

Reporte técnico de experimentos

Detección de tráfico de red malicioso mediante clasificación
con desbalance extremo de clases e interpretabilidad

Proyecto Aplicado en Analítica de Datos — Entrega final
Anexo técnico (ii): implementación y experimentos del prototipo

Mariana Pérez
Bryan Rodriguez
Daniela Zúñiga

[[PENDIENTE: fecha de entrega]]

Resumen

Este reporte documenta la implementación y los experimentos del prototipo de detección de tráfico de red malicioso sobre el conjunto de datos CIC-IDS2017. Se describen: (i) los modelos seleccionados y su correspondencia con los requerimientos aprobados; (ii) las alternativas evaluadas con sus ventajas y desventajas; (iii) la verificación de los supuestos de cada familia de modelos y el tratamiento previo de los datos que cada una exige; (iv) el protocolo de entrenamiento, validación y parametrización, con la justificación de las métricas empleadas; (v) el análisis de resultados y los ajustes implementados a partir de él; y (vi) el estado final de implementación del prototipo, con las pruebas que verifican cada requerimiento, sus riesgos y las próximas iteraciones. El modelo seleccionado alcanza un macro-F1 de 0,975 sobre el conjunto de prueba, manteniendo las falsas alarmas en 0,12 % del tráfico benigno; la estimación conservadora, que no depende del puerto de destino, es de 0,942.

1. Contexto y alcance

Problema: Un centro de operaciones de seguridad (SOC) recibe cientos de miles de *flujos de red* por día. Un flujo es una conversación entre dos computadores resumida en medidas numéricas (cuántos paquetes, de qué tamaño, con qué ritmo, cuánto duró), sin acceso al contenido de la comunicación. Revisarlos manualmente es inviable y las reglas por firma no reconocen ataques nuevos.

Usuario objetivo: Analista de seguridad de un SOC. Conoce redes y tipos de ataque, pero no es científico de datos ni programador, y sobre todo se enfrenta a un volumen de tráfico que ninguna persona puede revisar manualmente. Necesita *priorizar* qué flujos investigar y disponer de una razón que pueda justificar ante terceros.

Datos: CIC-IDS2017 (Canadian Institute for Cybersecurity; Sharafaldin et al., 2018): una semana simulada de tráfico corporativo, con 2.830.743 flujos etiquetados como benignos (80,3 %) o como uno de 14 tipos de ataque (19,7 %). El desbalance es extremo: la clase más frecuente supera a la más rara en una razón de 206.645 a 1.

Encuadre analítico: El problema se aborda como uno de **clasificación con desbalance de clases e interpretabilidad**, no como un problema de seguridad ofensiva.

2. Modelos propuestos y su correspondencia con los requerimientos

2.1. Requerimiento al modelo

Los requerimientos aprobados se agrupan en tres preguntas analíticas, y cada una determina una familia de modelos distinta, donde el Cuadro 1 hace esa correspondencia explícita.

Cuadro 1: Correspondencia entre requerimientos, pregunta analítica y modelo.

Req.	Requerimiento	Pregunta analítica	Modelo que responde
R1, R6, R14	Clasificar el tipo de ataque pese al desbalance, con desempeño accionable	¿De qué tipo es este ataque?	Clasificador multiclase supervisado (11 clases)
R2	Reducir falsas alarmas para disminuir la fatiga de alertas	¿Cuántas alarmas falsas genera?	El mismo multiclase, evaluado por precisión y recall por clase
R3	Explicar en qué se basa el modelo	¿Por qué se marcó esta alerta?	Técnicas de interpretabilidad aplicadas al clasificador: lectura global e individual
R4	Detectar comportamientos anómalos no etiquetados	¿Esto es raro, aunque no sepa qué es?	Detector de anomalías no supervisado
R5	Demostrar valor frente al escenario sin herramienta	¿Cuánto trabajo ahorra?	Comparación contra clasificador trivial y contra revisión manual

2.2. Integración de los tres modelos

La solución no es un modelo sino **tres modelos que se complementan**, porque ninguno cubre solo todo el espacio del problema. La Figura 1 muestra cómo se integran.

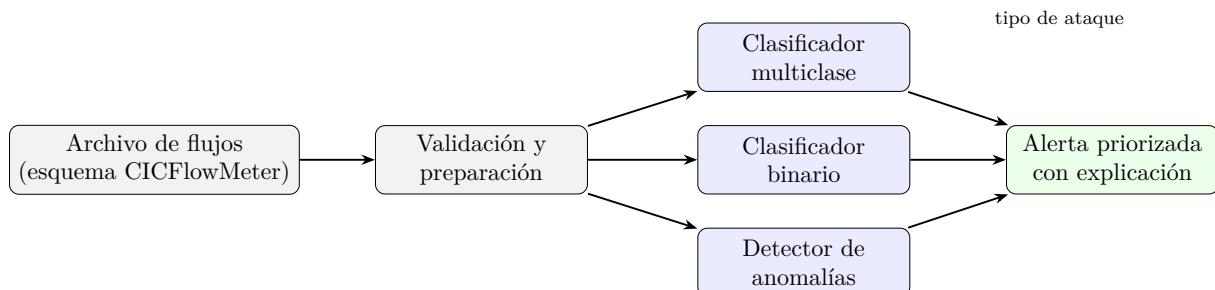


Figura 1: Arquitectura de la solución: los tres modelos operan sobre el mismo flujo preparado y sus salidas se combinan en una única alerta. El multiclase aporta el tipo de ataque; el binario, un veredicto robusto de ataque/no ataque que cubre las clases que el multiclase no pudo aprender; el detector de anomalías, comportamientos raros sin necesidad de etiquetas.

La razón del uso de tres modelos es una limitación de los datos que se documenta más adelante (Sección 7) y son estos dos tipos de ataque: Heartbleed (11 casos) e Infiltration (36 casos), tienen tan pocos casos que no pueden aprenderse como clases propias. El clasificador binario y el detector de anomalías cubren ese vacío desde ángulos distintos.

Qué se despliega y qué queda en el análisis: No todo lo que realizamos hace parte del artefacto (por practicidad), lo separamos en dos categorías para evitar confusión:

- **Modelos desplegados en el artefacto (tres).** El clasificador multiclase y el clasificador binario, ambos *predictivos* (asignan una etiqueta a cada flujo); y el detector de anomalías, *descriptivo*, ya que no predice una etiqueta conocida sino que señala desviaciones respecto del patrón de tráfico normal.

- **Herramientas de análisis e interpretación (no desplegadas como modelos).** La regresión logística, usada como referencia interpretable para verificar el supuesto de linealidad; el Local Outlier Factor, usado como contraste del detector de anomalías; y las técnicas de interpretabilidad (importancia por permutación y SHAP), que son *descriptivas*: no producen una predicción propia sino que explican la del clasificador.

3. Alternativas evaluadas: ventajas y desventajas

Para cada requerimiento se plantearon alternativas y se llevaron a un plan de pruebas comparativo. El Cuadro 2 resume el análisis previo, la Sección 6 reporta cuál ganó y por qué.

Cuadro 2: Alternativas consideradas por requerimiento, con ventajas y desventajas.

Req.	Alternativa	Ventajas	Desventajas
R1	Regresión logística	Coeficientes directamente legibles; entrenamiento simple; sirve de referencia honesta	Supone frontera lineal; exige escalado y control de colinealidad
R1	HistGradient-Boosting	Captura relaciones no lineales; no requiere escalado; muy rápido con datos tabulares grandes	Menos transparente: exige técnicas externas de interpretación
R1	Sin corrección de desbalance	No introduce sesgo artificial; sirve como control	Las clases minoritarias quedan ignoradas
R1	Pesos de clase	Barato; no fabrica datos; ataca la función de pérdida	Si el modelo no puede separar, solo multiplica falsas alarmas
R1	SMOTE (sobremuestreo sintético)	Densifica la clase rara donde ya existe	No crea información nueva; inútil con muy pocos casos
R1	Submuestreo + SMOTE	Reduce costo de cómputo; equilibra el conjunto	Descarta información real; resultó inestable
R4	Isolation Forest	Escala bien; sin etiquetas; umbral interpretable por cuantiles	Ciego a ataques cuya anomalía está en el agregado
R4	Local Outlier Factor	Densidad local en lugar de aislamiento: ve la fuerza bruta contra SSH, que el Isolation Forest no detecta (recall 0,91 frente a 0,00)	Costo de predicción alto; pierde Heartbleed (0,00 frente a 0,89) y los DoS lentos (slowloris 0,00 frente a 0,52); su recall global es un tercio del elegido (0,035 frente a 0,118)
R3	Importancia por permutación	Agnóstica al modelo; se mide sobre datos no vistos	Solo explica el comportamiento global, no la alerta individual
R3	SHAP (TreeExplainer) (Lundberg y Lee, 2017)	Explica cada alerta con fundamento matemático; 2 ms por flujo	Añade dependencias pesadas (≈ 100 MB de memoria)
R3	Desviación vs. rango benigno	Sin dependencias; muy simple de explicar	Describe qué es raro, no qué miró el modelo

En **negrilla**, la alternativa seleccionada para cada requerimiento tras las pruebas comparativas de la Sección 6.

Diseño del plan de pruebas. Las alternativas de R1 se cruzaron en una **matriz de dos ejes** (2 familias de modelo $\times$ 4 técnicas de desbalance = 8 combinaciones), evaluadas todas bajo condiciones idénticas. El diseño no es casual: separar los ejes permite *atribuir cada mejora a su causa* y distinguir si un problema se debe al desbalance o a la capacidad del modelo. Como se verá, esa distinción resultó ser el hallazgo central del proyecto.

4. Verificación de supuestos y tratamiento previo de los datos

Cada familia de modelos impone supuestos distintos sobre los datos. El Cuadro 3 documenta qué se verificó y qué tratamiento se aplicó en consecuencia.

Cuadro 3: Supuestos por familia de modelo, verificación realizada y tratamiento aplicado.

Modelo	Supuesto	Verificación	Tratamiento
Regresión logística	Separabilidad lineal de las clases	Curvas precision-recall por clase: Bot y Web Attack quedan en AP de 0,22 y 0,30, indicando que no son linealmente separables	Se conserva como referencia interpretable; se añade un modelo no lineal para las clases afectadas
Regresión logística	Variables no redundantes (baja colinealidad)	45 pares con $ r > 0,95$ sobre el conjunto de entrenamiento, agrupados en 13 bloques (Apéndice A)	Poda a una representante por bloque: 71 $\rightarrow$ 48 variables (Cuadro A.1 y Figura A.1)
Regresión logística	Variables en escala comparable	Rangos heterogéneos (microsegundos frente a conteos)	StandardScaler ajustado <i>dentro</i> del pipeline de validación cruzada, solo con el tramo de entrenamiento de cada partición
Isolation Forest	Variables en escala comparable	Los mismos rangos heterogéneos; aquí no hay validación cruzada	StandardScaler ajustado <i>únicamente</i> con el tráfico benigno del lunes (394.236 flujos), el mismo con el que se entrena el detector
HistGradient-Boosting	Ninguno sobre escala o linealidad	Se verifica que el desempeño no dependa del escalado	No se escala; se conserva la escala original
Isolation Forest	Las anomalías son pocas y estructuralmente distintas	Distribución de scores sobre tráfico conocido-benigno	Umbral fijado por cuantiles de esa distribución (Sección 5.1)
Todos	Ausencia de valores no finitos	2.867 filas (0,10 %) con Inf/NaN en dos variables de tasa (divisiones por duración cero)	Eliminadas (1.563 de ellas en este paso; el resto ya había salido con los duplicados), con verificación automática de que no quedan
Todos	Los valores numéricos representan magnitudes	El valor -1 en Init_Win_bytes no es una magnitud sino un código de “no aplica” (51 % de las filas en Init_Win_bytes_backward y 35 % en Init_Win_bytes_forward)	Se convierte en indicador binario y el -1 pasa a 0
Todos	Independencia entre observaciones	330.995 filas duplicadas exactas (11,69 %)	Eliminadas <i>antes</i> de partir en entrenamiento y prueba
Todos	Variables informativas	8 variables constantes (varianza cero)	Eliminadas

4.1. El supuesto de linealidad verificado

La verificación más consecuente fue la de linealidad. La Figura 2 muestra las curvas precision-recall de las dos clases problemáticas con tres configuraciones. Con el modelo lineal, ninguna técnica de desbalance las rescata, es decir, el problema no era la proporción de clases, sino que **sus fronteras de decisión no son lineales**. Al sustituir la familia de modelo (sin ninguna corrección de desbalance) ambas mejoran sustancialmente. Este resultado es lo que motivó el

ajuste principal que se realizó en la Sección 7.

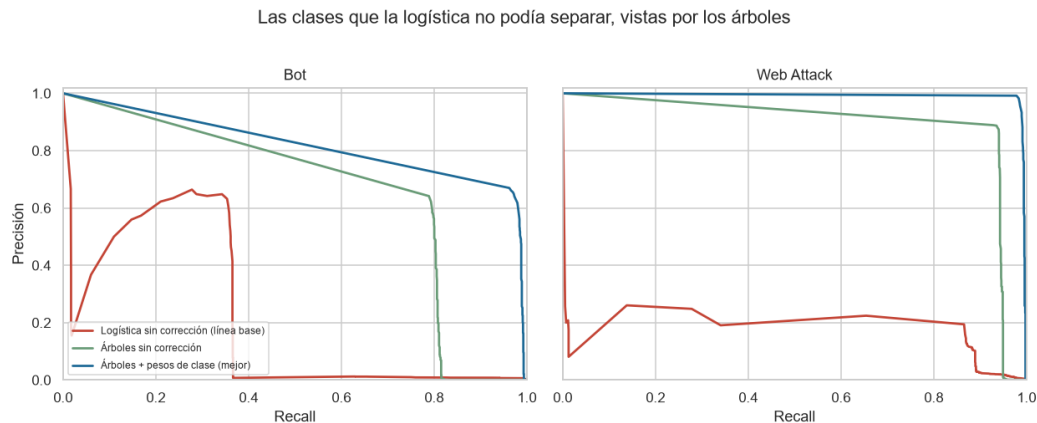


Figura 2: Verificación del supuesto de linealidad. Las curvas se desplazan hacia la esquina superior derecha al cambiar de familia de modelo, no al cambiar la técnica de desbalance. Esta es la evidencia de que el supuesto de separabilidad lineal no se cumple para estas clases.

5. Entrenamiento, validación, parametrización y métricas

5.1. Protocolo experimental

- **Partición única:** 80 % entrenamiento (1.998.462 flujos) y 20 % prueba (499.616), estratificada por clase para preservar las proporciones incluso en las clases más raras. El conjunto de prueba se apartó una sola vez y **no se utilizó** durante todo el desarrollo.
- **Validación cruzada estratificada de 5 particiones** sobre el entrenamiento para la comparación de alternativas, la selección de modelo y el ajuste del punto de operación. Se reporta media $\pm$ desviación entre particiones. Una excepción, por costo computacional: la *importancia por permutación* se midió con una sola partición —modelo ajustado con el 80 % de la primera y medido sobre 300.000 flujos de su validación— (Cuadro B.8).
- **Selección de variables:** 71 $\rightarrow$ 48, decidida *únicamente* con el conjunto de entrenamiento (Cuadro 3; evidencia en el Apéndice A).
- **Control de fuga de información:** deduplicación previa a la partición; escalado y remuestreo *dentro* del pipeline de validación, de modo que solo se ajustan con el tramo de entrenamiento de cada partición.
- **Reproducibilidad:** semilla fija (42) en todo el proceso y versiones exactas de dependencias.

Parametrización: El modelo de árboles se configura con un máximo de 200 iteraciones (`max_iter=200`), ponderación balanceada de clases y semilla 42; el resto de sus parámetros queda en los valores por defecto de la librería. Con más de 10.000 observaciones, scikit-learn activa automáticamente la **parada temprana** (`early_stopping='auto'`): reserva el 10 % del tramo de entrenamiento como validación interna y detiene el entrenamiento cuando la pérdida sobre esa validación deja de mejorar durante 10 iteraciones seguidas. Por eso 200 es un *techo*, no el número de iteraciones usadas: el clasificador multiclase final se detuvo en **43 iteraciones** (473 árboles, uno por clase y por iteración) y el binario en **110**, cifras leídas del atributo `n_iter_` de los modelos serializados (Apéndice B). SMOTE eleva a 20.000 casos las clases con menos de ese umbral (igualarlas a la clase mayoritaria habría exigido fabricar ≈ 14 millones de filas sintéticas); el submuestreo reduce la clase benigna a 200.000 casos antes de aplicar SMOTE.

Cómo se fija el umbral del detector de anomalías (cuantiles del lunes): El Isolation Forest no produce una etiqueta sino un *score* de qué tan normal se ve cada flujo; para convertirlo en una decisión hace falta un corte, y ese corte se fija en cuatro pasos. (1) El detector se entrena únicamente con el tráfico del **lunes**, el único día de la semana sin ataques (394.236 flujos benignos del conjunto de entrenamiento), con el escalado ajustado sobre ese mismo tráfico. (2) Luego se le pide puntuar ese mismo tráfico conocido-normal, lo que produce una distribución empírica de los *scores* de flujos que *sabemos* normales. (3) El umbral se fija en un percentil de esa distribución (el 1 % como referencia; 0,5 % y 2 % como alternativas que el tablero permite elegir), de modo que, por construcción, exactamente ese porcentaje del tráfico conocido-normal queda por debajo del corte. (4) Cualquier flujo de otro día cuyo *score* caiga por debajo del corte se marca como anómalo. La ventaja es que el umbral no es un número arbitrario sino un **presupuesto explícito de falsas alarmas**: elegir el 1 % significa “acepto equivocarme en 1 de cada 100 flujos normales”. El valor numérico del corte (por ejemplo, $-0,614$ al 1 %) no tiene significado por sí mismo y por eso no se reporta como parámetro sino como consecuencia del percentil elegido. Fijarlo así reconoce además la advertencia de la literatura de que el “benigno” del lunes puede contener ataques sin etiquetar: el percentil asume explícitamente cierta contaminación en lugar de suponer un lunes perfectamente limpio. Y es lo que permite medir la deriva (R11): si en otro día la fracción de tráfico normal que cae bajo el corte se aleja del presupuesto, el tráfico normal cambió y el detector debe recalibrarse.

Calibración de hiperparámetros: No se realizó una búsqueda exhaustiva de hiperparámetros, de forma deliberada. La configuración exacta con la que se entrenó cada modelo —qué se fijó explícitamente y qué quedó en el valor por defecto de la librería, leída de los modelos serializados y del código, no reconstruida de memoria— se reporta parámetro por parámetro en el Apéndice B. Con esa configuración el modelo ya superaba el requerimiento de desempeño por más de tres puntos de macro-F1, y la matriz de la Sección 6 muestra que la variación atribuible a la familia de modelo y a la técnica de desbalance (de 0,543 a 0,970) domina ampliamente cualquier ganancia esperable de un ajuste fino de parámetros. El esfuerzo de calibración se dirigió, en cambio, donde el análisis mostró margen real: el *punto de operación* de la clase Bot (Sección 7, ajuste 4), fijado con datos de entrenamiento exclusivamente. Una búsqueda adicional habría añadido riesgo de sobreajustar al conjunto de validación a cambio de una ganancia marginal.

5.2. Justificación de las métricas

La elección de métricas es clave en problemas con este nivel de desbalance:

- **Exactitud (accuracy): descartada.** Con 83 % de tráfico benigno en el conjunto ya depurado (80,3 % en los datos crudos: la deduplicación redujo más a los ataques que al tráfico normal), un clasificador que responda siempre “normal” alcanza 83 % de exactitud sin detectar un solo ataque. Se verificó empíricamente: ese clasificador trivial obtiene macro-F1 de 0,082 en el problema multiclase.
- **Macro-F1: métrica principal.** Promedia el F1 de todas las clases con igual peso, de modo que una clase con 1.500 casos pesa lo mismo que una con dos millones. Es la métrica adecuada cuando las clases raras son las que importan.
- **Precisión y recall por clase: obligatorios.** El promedio puede ocultar que una clase concreta no se detecta. En el contexto del SOC tienen lectura directa: el recall es la fracción de ataques encontrados; la precisión, la fracción de alarmas que no son falsas (fatiga de alertas).
- **Curvas precision-recall y AP en lugar de ROC/AUC.** La tasa de falsos positivos que usa la curva ROC tiene por denominador la clase mayoritaria: con dos millones de flujos benignos, 40.000 falsas alarmas son apenas un 2 % y la curva luce excelente, mientras la precisión real se desploma. La curva PR expone ese costo. Además, el AP resultó una

herramienta *de diagnóstico*: un AP alto con recall bajo indica un problema de umbral, mientras que un AP bajo indica que el modelo no separa la clase.

- **Macro-F1 restringido a clases de ataque:** se reporta junto al global como cifra más exigente, al excluir del promedio la clase benigna.

5.3. Líneas base

Antes de experimentar se fijaron dos tipos de referencia (baseline) contra los cuales medir cualquier mejora posterior (Cuadro 4). La primera es el **clasificador trivial**: un modelo que responde siempre la clase mayoritaria sin mirar los datos. Marca lo que sería el piso absoluto, porque cualquier modelo que no lo supere no estaría aprendiendo nada. La segunda es la **regresión logística sin corrección de desbalance**, que representa la solución más simple y transparente aplicable al problema. Ambas se calcularon con el mismo protocolo de validación cruzada que las alternativas posteriores, de modo que todas las cifras del reporte son comparables entre sí.

Cuadro 4: Líneas base (validación cruzada sobre el conjunto de entrenamiento).

Configuración de referencia	macro-F1
Binaria — clasificador trivial (siempre “normal”)	0,453
Binaria — regresión logística	0,921
Multiclase — clasificador trivial (siempre “benigno”)	0,082
Multiclase — regresión logística	0,701

6. Análisis de resultados

6.1. Comparación de alternativas

El Cuadro 5 presenta las ocho combinaciones evaluadas bajo condiciones idénticas, y la Figura 3 las ordena visualmente.

Cuadro 5: Matriz de alternativas: macro-F1 (media $\pm$ desviación entre las 5 particiones).

Modelo	Sin corrección	Pesos de clase	SMOTE	Submues. + SMOTE
Regresión logística	0,700 $\pm$ 0,014	0,543 $\pm$ 0,004	0,832 $\pm$ 0,005	0,650 $\pm$ 0,002
Árboles (HistGB)	0,948 $\pm$ 0,008	0,970 $\pm$ 0,002	0,966 $\pm$ 0,003	0,937 $\pm$ 0,046

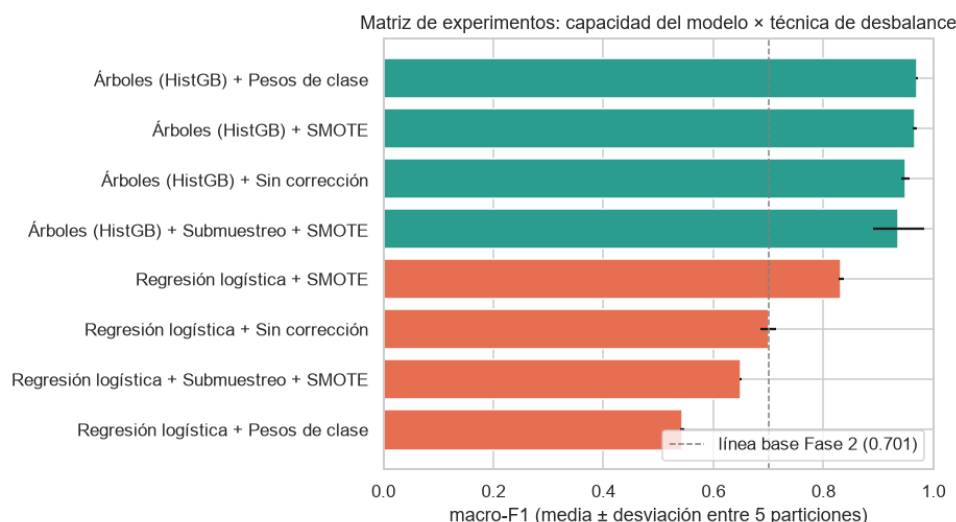


Figura 3: Las ocho combinaciones ordenadas por desempeño. Todas las configuraciones basadas en árboles superan a todas las lineales.

Encontramos tres hallazgos a partir de la matriz:

1. **La capacidad del modelo pesó más que la técnica de desbalance.** La peor configuración de árboles (0,937) supera a la mejor configuración lineal (0,832). Esto nos indicó que más que “falta corrección de desbalance” era “falta capacidad de representación”.
2. **Reponderar sin capacidad es contraproducente.** Los pesos de clase *hunden* a la regresión logística (0,700 → 0,543): elevan el recall de las clases minoritarias a $\approx 0,99$ pero con precisión de 0,02–0,03, lo que equivale a ≈ 264.000 flujos benignos marcados como ataque. Con un modelo capaz, la misma técnica es la mejor (0,948 → 0,970).
3. **La estabilidad importa tanto como la media.** La combinación submuestreo + SMOTE se descartó no por su media (0,937) sino por su desviación (0,046): una de las particiones cayó a 0,846.

6.2. Modelo seleccionado para cada requerimiento

Con la evidencia anterior se fijó qué modelo responde a cada requerimiento aprobado del prototipo. Los códigos son los mismos del Cuadro 1, y el enunciado de cada uno se repite aquí para facilitar la lectura.

- **R1, R6 y R14** — clasificar el tipo de ataque pese al desbalance, con desempeño accionable: **HistGradientBoosting con pesos de clase**.
- **R3** — explicar en qué se basa el modelo: **importancia por permutación** para la lectura global y **SHAP** para la explicación de cada alerta individual. La regresión logística se conserva como lectura auxiliar del análisis, no como componente del artefacto.
- **R4** — detectar comportamientos anómalos no etiquetados: **Isolation Forest** entrenado únicamente con tráfico benigno. El Local Outlier Factor se documenta como contraste y no se despliega.

7. Ajustes implementados a partir de los resultados

El análisis condujo a cinco ajustes, tres de ellos sobre la propia definición del problema de analítica:

1. Ajuste al esquema de solución: de un modelo a tres. Consecuencia directa del hallazgo de linealidad y de la escasez de dos clases. Ningún modelo único cubre el problema completo (Figura 1).

2. Ajuste al problema de analítica: agrupación de clases: Los tres ataques web (1.470, 652 y 21 casos) se agrupan en la familia “Web Attack”. La agrupación es la natural del dominio: los tres atacan la aplicación web.

3. Ajuste al problema de analítica: exclusión honesta de clases: Heartbleed (11 casos) e Infiltration (36) se excluyen del problema multiclase, ya que con validación cruzada de 5 particiones aportarían dos a siete casos por partición y cualquier métrica por clase sería ruido. Se evalúan en el nivel binario y con el detector de anomalías, y se reportan siempre acompañadas de su tamaño de muestra. Declarar la limitación es preferible a publicar una cifra que el dato realmente no sostiene.

4. Ajuste al punto de operación: umbral para la clase Bot. La clase Bot alcanzaba precisión de 0,614 con recall de 0,980, es decir cuatro de cada diez alarmas eran falsas. Su AP de 0,91 indicaba margen para intercambiar recall por precisión. Se fijó, *con datos de entrenamiento exclusivamente*, la regla de emitir la alarma de Bot solo cuando la probabilidad supere 0,999. Resultado sobre el conjunto de prueba: precisión 0,947 con recall 0,728.

5. Verificación de validez: la prueba del puerto de destino. La segunda variable más influyente resultó ser el puerto del servicio contactado, un *identificador* y no un comportamiento: en el conjunto de datos cada ataque usa siempre su puerto canónico, algo que en una red real no necesariamente tiene que pasar. Se reentrenó el modelo sin esa variable (Figura 4): el desempeño pasó de 0,970 a 0,950 en validación cruzada y de 0,967 a 0,942 en prueba. La caída es pequeña y el recall se mantiene en todas las clases, lo que indica que el modelo aprende comportamiento, y el costo se concentra en la precisión de las dos clases difíciles. Por honestidad metodológica, ambas cifras se reportan juntas y la variante sin puerto se declara como la **estimación conservadora**.

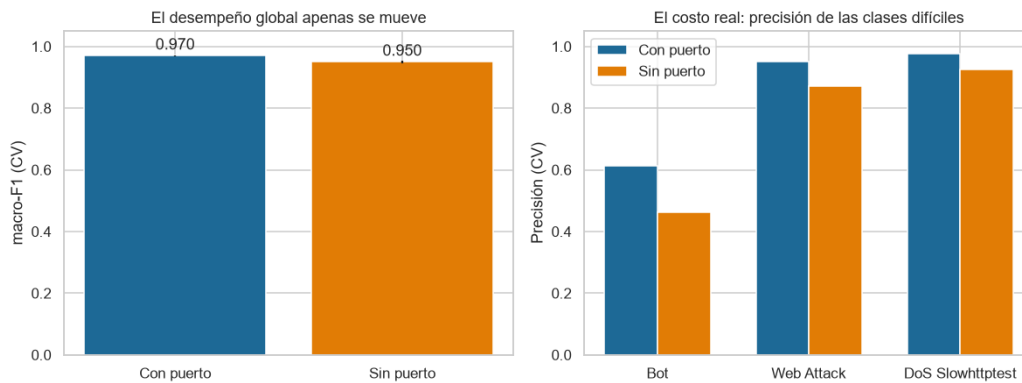


Figura 4: Prueba del puerto de destino. El desempeño global apenas se mueve al retirar el identificador; el costo se localiza en la precisión de las clases difíciles.

7.1. Resultados sobre el conjunto de prueba

Con el modelo ya definido, se utilizó el conjunto de prueba por primera y única vez (para tener los resultados como una medición completamente limpia).

Cuadro 6: Evaluación final sobre el conjunto de prueba (499.616 flujos).

Configuración	macro-F1	Solo ataques
Modelo final (con puerto + umbral de Bot)	0,975	0,972
Con puerto, regla estándar	0,967	0,964
Estimación conservadora (sin el puerto)	0,942	0,937

Validez del protocolo: Las cifras de prueba coinciden con las de validación cruzada ($0,970 \pm 0,002$ frente a $0,967$ con puerto; $0,950 \pm 0,004$ frente a $0,942$ sin él). Esa coincidencia es la evidencia de que no hubo sobreajuste ni fuga de información.

Cumplimiento de requerimientos: R6 exigía $\text{macro-F1} \geq 0,94$: se obtuvo $0,975$ ($0,942$ en la estimación conservadora). R2 exigía falsas alarmas $\leq 0,2\%$ sobre tráfico benigno: se obtuvo $0,12\%$. R5 exigía superar sustancialmente el clasificador trivial: $0,975$ frente a $0,082$. La segunda parte de R5, la reducción de la carga de revisión, se materializa en el panel de resumen del tablero, que la calcula sobre el archivo que carga el usuario.

Detección no supervisada (R4): El Isolation Forest, entrenado únicamente con tráfico benigno, detecta 89% de los flujos de Heartbleed, 52% de DoS slowloris y 48% de Infiltration (Heartbleed e Infiltration son justamente las familias que el modelo supervisado no pudo aprender) con un presupuesto de 1% de falsas alarmas (umbral por cuantiles del lunes, Sección 5.1). Su recall global es de $11,8\%$, y se documenta explícitamente su punto ciego: es incapaz de detectar ataques camuflados (fuerza bruta, escaneo de puertos, botnet), porque en ellos cada flujo individual parece normal y lo anómalo reside precisamente en el agregado. El recall superior a $0,4$ en las familias estructuralmente raras se cumple como exigía el R4.

Deriva temporal (R11): La tasa de falsas alarmas del detector se mantuvo en torno al 1% previsto casi todos los días, con una excepción documentada: un tramo del viernes alcanzó $9,6\%$. Se reporta como señal de que un despliegue real exigiría recalibración periódica.

8. Estado de implementación del prototipo

8.1. Componentes implementados

Esta sección resume el estado final del prototipo que se entrega. Todo lo que se describe está en el repositorio del proyecto, y lo que depende del tablero desplegado o de una persona ajena al equipo se marca como pendiente.

- **Modelos serializados.** Los tres modelos ($1,3\text{MB}$ en total) están en `models/` con sus metadatos: las 48 variables, las 11 clases, la regla de operación de Bot y los umbrales del detector. Volver a entrenar el detector con la semilla 42 reproduce sus umbrales decimal a decimal.
- **Tablero web de seis pantallas** (Streamlit): Panel de resumen, Clasificación, Detección de anomalías, Interpretabilidad, Alertas y Reportes. Valida el archivo antes de clasificar y explica en lenguaje llano lo que encuentra, filtra la cola de alertas por tipo de ataque, confianza y marca de anomalía, exporta en CSV y lleva un historial de la sesión. La pantalla Detección de anomalías incluye la tabla de falsas alarmas del detector por día, que sigue al presupuesto elegido y hace visible la deriva descrita en la Sección 7.
- **Dos demos integradas.** La barra lateral trae dos botones que cargan muestras del conjunto de prueba, es decir, de flujos que los modelos no vieron al entrenar. La *Demo rica en ataques* (499 flujos, $39,9\%$ de ataque y 14 tipos) muestra todo lo que el sistema detecta, y la *Demo de proporción realista* (500 flujos, $5,0\%$ de ataque y 10 tipos) muestra cuánto trabajo de revisión ahorra. Sus etiquetas verdaderas están aparte, en `data/demo/`, con

la misma numeración de filas del tablero, para comprobar los resultados fila por fila; el tablero no las lee.

- **Pruebas de los requerimientos.** La carpeta `tests/` tiene 25 pruebas, 19 automatizadas y 6 manuales, y cada una corresponde a la “prueba prevista” de un requerimiento de la tabla aprobada. Los scripts de prueba originales se perdieron porque vivían fuera del repositorio, así que los reconstruimos a partir de la tabla de requerimientos. Se ejecutan con un solo comando (`python -m pytest -v`) y en el repositorio entregado dan 17 pasadas y 8 omitidas: las 6 manuales, que dependen del despliegue o de una persona, y 2 que necesitan los datos procesados del análisis. Cada omisión explica su razón en la salida.
- **Tiempo de respuesta en local.** Para R9 (tiempo de respuesta y volumen soportado), 50.000 flujos se validan y clasifican en menos de un segundo (entre 0,7 y 0,8 s en mediciones repetidas) con un pico de memoria cercano a 390 MB, frente a la promesa de 30 segundos y 1 GB disponible.
- **Documentación de la entrega.** El manual de usuario, que es el documento principal, y como anexos el diagrama esquemático, este reporte y la tabla de requerimientos diligenciada, con el resultado, el estado y la evidencia de cada requerimiento.
- **Licencias.** Las dependencias directas del tablero son de código abierto con licencias permisivas, leídas de los metadatos de cada paquete: Apache-2.0 (streamlit), BSD-3-Clause (scikit-learn, pandas y joblib), MIT (shap y pytest) y, en numpy, BSD-3-Clause con componentes 0BSD, MIT, Zlib y CC0-1.0. Todas son compatibles con el uso académico del proyecto.

Ajustes implementados frente al mockup. El tablero entregado difiere del mockup aprobado en tres puntos, decididos a partir de lo que el prototipo puede sostener:

1. La columna de severidad se reemplazó por la marca de anomalía, porque no hay una escala de severidad justificada por criterio de dominio.
2. La exportación en PDF se dejó solo en CSV, porque el CSV cubre la necesidad y mantiene liviana la herramienta.
3. El historial trazable quedó como historial de la sesión, porque el alojamiento gratuito no garantiza persistencia entre reinicios.

Despliegue. El tablero se publica en `[[PENDIENTE: URL]]`. Las mediciones sobre el tablero desplegado quedan `[[PENDIENTE: tiempo de 50.000 flujos, volumen máximo estable, reanudación tras inactividad y apertura en Chrome, Firefox y Edge]]`, y la prueba de usabilidad con una persona ajena al proyecto queda `[[PENDIENTE: resultado de la prueba de usabilidad y de la revisión de textos]]`.

8.2. Pendientes de la entrega y próximas iteraciones

Lo que queda pendiente depende del despliegue o de personas ajenas al equipo. El Cuadro 7 lo lista con el requerimiento que lo motiva, su responsable y su fecha.

Cuadro 7: Pendientes de la entrega, con responsable y fecha.

Pendiente	Requerimiento o entregable que lo motiva	Responsable	Fecha
Despliegue y acceso desde otro equipo	R10, disponibilidad del prototipo: accesible sin instalación y con reanudación en menos de 1 minuto	[[PENDIENTE: responsable]]	[[PENDIENTE: fecha]]
Tiempo de respuesta en el desplegado	R9, tiempo de respuesta y volumen soportado: hasta 50.000 flujos en menos de 30 segundos, medido con el lote que genera <code>tests/generar_lote_r9.py</code>	[[PENDIENTE: responsable]]	[[PENDIENTE: fecha]]
Apertura en Chrome, Firefox y Edge	R22, compatibilidad de acceso con los navegadores de escritorio de uso común	[[PENDIENTE: responsable]]	[[PENDIENTE: fecha]]
Prueba de usabilidad	R20, operación sin conocimientos técnicos: una persona ajena completa cargar, revisar alertas y exportar sin asistencia	[[PENDIENTE: responsable]]	[[PENDIENTE: fecha]]
Revisión de textos por una persona ajena	R21, comprensibilidad de los resultados: cada pantalla y métrica en lenguaje llano	[[PENDIENTE: responsable]]	[[PENDIENTE: fecha]]
Capturas de pantalla	Manual de usuario (entregable principal)	[[PENDIENTE: responsable]]	[[PENDIENTE: fecha]]
Presentación ejecutiva	Entregable: siete minutos con el problema, el esquema de solución, la demostración y la propuesta de valor	[[PENDIENTE: responsable]]	[[PENDIENTE: fecha]]

Próximas iteraciones. Lo que la tabla de requerimientos dejó fuera de alcance en esta iteración, junto con las acciones correctivas de los requerimientos que no se cumplen del todo, define las siguientes:

- **Detección no supervisada de ataques camuflados**, con características agregadas por ventana de tiempo, porque el flujo individual no contiene la señal.
- **Monitoreo continuo de la deriva** en operación, para R11 (monitoreo de desempeño y re-entrenamiento): medir periódicamente las falsas alarmas por día y recalibrar el detector cuando se alejen del presupuesto.
- **Calibración de probabilidades** (isotónica), que resolvería la fragilidad del umbral de operación de Bot observada en la evaluación final.
- **Integración con el SIEM** mediante una API de scoring, para R19 (integración con sistemas del SOC); en esta iteración la vía de integración es la exportación en CSV.
- **Historial persistente entre sesiones y exportación en PDF**, que requieren un alojamiento con persistencia y librerías adicionales.
- **Clasificación de severidad de alertas**, cuando exista una escala definida y justificada por criterio de dominio.
- **Ingesta de tráfico en tiempo real**, porque el prototipo opera por lotes sobre archivos cargados por el usuario.
- **Clasificación por subtipo de ataque web** (XSS y SQL Injection), cuando haya casos suficientes para métricas por clase confiables (hoy son 652 y 21).

8.3. Riesgos identificados

- **Deriva temporal del tráfico normal.** El detector se calibró con el lunes y, con un presupuesto de 1 %, un tramo del viernes llegó a 9,6 % de falsas alarmas, frente a cerca de 1 % en los demás días. En una red real la deriva sería continua. Mitigación: el tablero

expone la tasa de falsas alarmas por día, y el detector se recalibra volviendo a entrenarlo con tráfico normal reciente.

- **Generalización fuera de CIC-IDS2017.** Es tráfico simulado de 2017 en el que cada ataque usa su puerto típico. La estimación conservadora sin el puerto de destino (0,942) es la más cercana a una red distinta, pero solo una evaluación con tráfico real permitiría afirmar el desempeño en producción. Además, el umbral de Bot (0,999) resultó frágil, porque en la variante sin puerto eliminó todas las alarmas de Bot. Mitigación: se reportan las dos cifras juntas y la calibración de probabilidades queda como próxima iteración.
- **Calidad del conjunto de datos.** Encontramos estudios que documentan errores de etiquetado en CIC-IDS2017 (Engelen et al., 2021; Rosay et al., 2022; Lanvin et al., 2023). Se declara como limitación, afecta principalmente al detector no supervisado, entrenado con el lunes supuestamente benigno, y se mitiga fijando el umbral por cuantiles, que asume explícitamente cierta contaminación. Además, Heartbleed (11 casos) e Infiltration (36) son tan escasos que sus cifras se reportan siempre como ilustrativas.
- **Sensibilidad a la versión de scikit-learn.** Los modelos serializados solo cargan de forma confiable con la versión exacta con que se generaron (1.9.0). Mitigación: la versión está fijada de forma idéntica en `requirements.in`, `requirements.txt` y `requirements-analisis.txt`, el tablero la verifica al arrancar y muestra una advertencia si no coincide, y los modelos se regeneran con `python -m src.modelo_final`. En un Mac con procesador Intel los entornos no se pueden instalar (SHAP depende de `numba`, que no publica versiones para esa plataforma con Python 3.13), así que allí se usa la URL pública.
- **Limitaciones del alojamiento gratuito.** No garantiza persistencia entre reinicios ni tiempos de reanudación acotados. Mitigación: el historial se declara como de sesión y las descargas en CSV permiten conservar los resultados. El tiempo de reanudación medido queda [[PENDIENTE: tiempo de reanudación tras inactividad en el desplegado]].

9. Conclusiones

1. El tipo de ataque puede clasificarse con alta calidad pese al desbalance extremo: macro-F1 de 0,975 sobre el conjunto de prueba, y 0,942 en la estimación conservadora que no depende del puerto de destino.
2. **El diagnóstico correcto importa más que la técnica.** Las clases que parecían imposibles no necesitaban más remuestreo sino un modelo capaz de representar fronteras no lineales. Reponderar sin capacidad multiplica las falsas alarmas.
3. Los ataques se distinguen por *comportamiento* (duración, ritmo entre paquetes, tamaños) y no solo por el puerto: retirar el único identificador cuesta entre dos y tres puntos de macro-F1.
4. El modelo no supervisado es un complemento, no un sustituto: detecta sin etiquetas justamente las familias que el supervisado no puede aprender, pero es ciego a los ataques camuflados.
5. El prototipo está implementado: un tablero de seis pantallas con dos demos integradas, tres modelos serializados y 25 pruebas ligadas a los requerimientos (19 automatizadas y 6 manuales). La Sección 8 resume su estado final, los pendientes que dependen del despliegue [[PENDIENTE: resultado de las verificaciones en el tablero desplegado]] y las próximas iteraciones.

Referencias

- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). *Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization*. 4th International Conference on Information Systems Security and Privacy (ICISSP), Portugal, enero de 2018.
- Engelen, G., Rimmer, V., & Joosen, W. (2021). *Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study*. IEEE Security and Privacy Workshops.
- Rosay, A., et al. (2022). *Network Intrusion Detection: A Comprehensive Analysis of CICIDS2017*. ICISSP.
- Lanvin, M., et al. (2023). *Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes*. CRiSIS 2022.
- Lundberg, S. M., & Lee, S. I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems (NeurIPS).

A. Evidencia de colinealidad: los 13 bloques de variables redundantes

La retroalimentación pidió anexar una lista o mapa de correlaciones que respalde la poda de 71 a 48 variables (Cuadro 3). Este apéndice la reproduce sobre los mismos datos con los que se tomó la decisión.

Cómo se calculó. Sobre una muestra de 500.000 flujos del **conjunto de entrenamiento** (semilla 42; el conjunto de prueba no participa) se calculó la correlación de Pearson en valor absoluto entre las 71 variables numéricas del dataset depurado. Hay **45 pares** con $|r| > 0,95$. Si cada variable es un nodo y cada par por encima del umbral es un arco, esos 45 pares forman **13 componentes conexas**: los *bloques* del Cuadro A.1, que abarcan 36 variables. De cada bloque se conserva **una representante**, elegida por interpretabilidad (el nombre más directo de explicar a un lector no experto en redes), y se eliminan las otras 23: $71 - 23 = 48$. La razón de la poda es que dos columnas casi idénticas se reparten la importancia en los modelos y ensucian la interpretación; para la regresión logística, además, violan el supuesto de baja colinealidad.

Cómo leer la figura. La Figura A.1 muestra el mapa de $|r|$ entre las 36 variables implicadas, ordenadas bloque a bloque (representante primero, marcada con $\star$), con cada bloque recuadrado y los valores anotados dentro de él. Dos precisiones: (i) un bloque es una componente conexa, así que dos variables del mismo bloque no tienen por qué superar 0,95 *entre sí* —por ejemplo, `Bwd Packet Length Mean` y `Bwd Packet Length Std` correlacionan 0,94, pero ambas superan 0,95 con `Bwd Packet Length Max`—; (ii) fuera de los recuadros hay correlaciones altas pero por debajo del umbral (tonos medios), que deliberadamente no motivaron eliminaciones. La figura del análisis exploratorio (`02_correlaciones_redundantes.png`) usaba $|r| > 0,999$ sobre una muestra del dataset completo y servía para ilustrar los alias exactos; la de este apéndice se calcula al umbral con el que realmente se decidió y solo sobre el entrenamiento, y coincide exactamente con el cuadro.

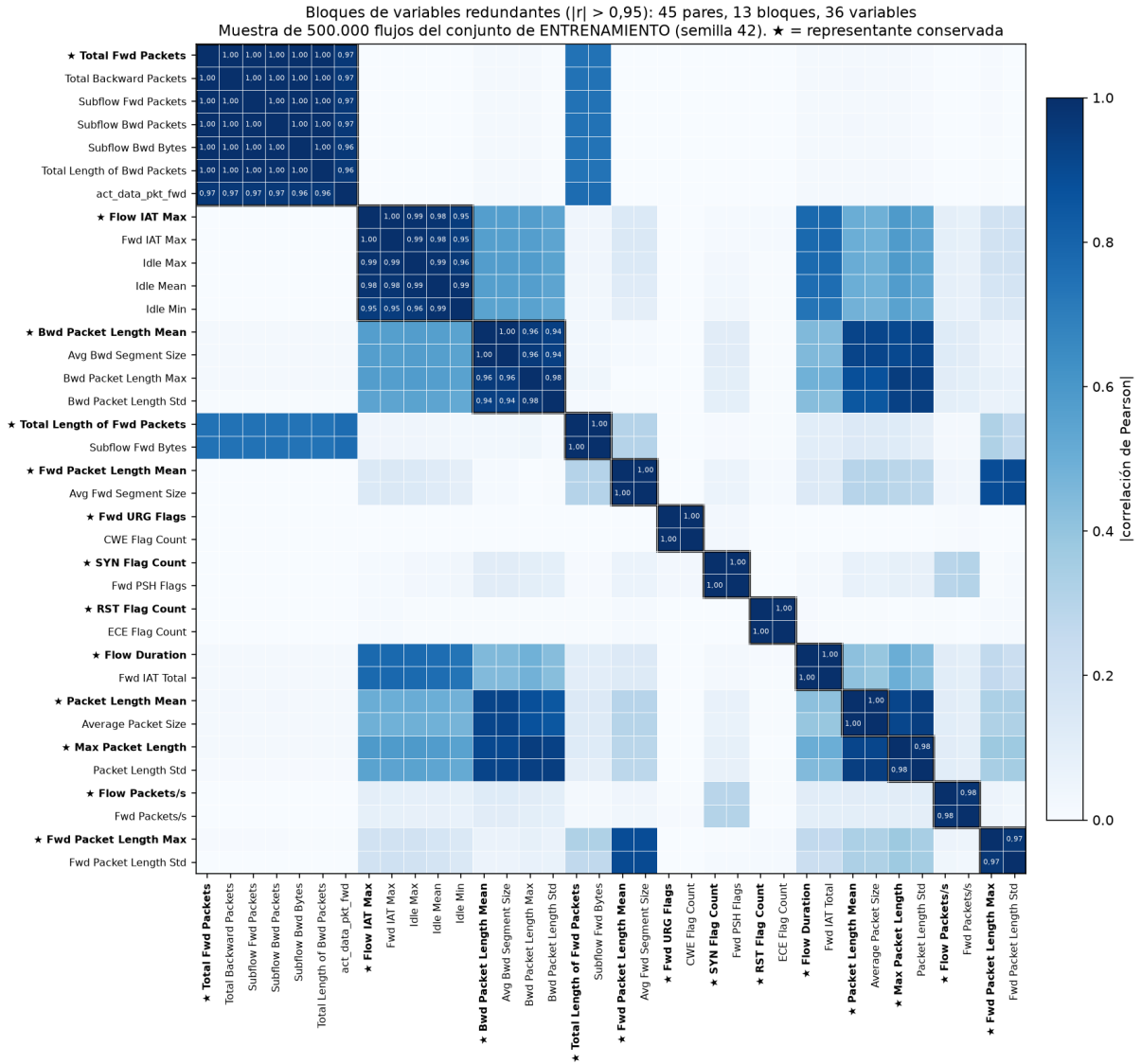


Figura A.1: Mapa de $|r|$ (correlación de Pearson en valor absoluto) entre las 36 variables que participan en algún par con $|r| > 0,95$, sobre 500.000 flujos del conjunto de entrenamiento (semilla 42). Cada recuadro negro es uno de los 13 bloques del Cuadro A.1; ★ marca la representante conservada. Los valores anotados son los de cada bloque; fuera de los recuadros ninguna pareja supera el umbral.

Reproducibilidad. Los bloques se re-derivan con `calcular_grupos()` en `src/features.py` y el script `reports/apendices/generar_apendices.py` verifica que coinciden uno a uno con la lista `GRUPOS_CORRELACIONADOS` antes de dibujar la figura y escribir el cuadro; el detalle numérico por bloque queda en `reports/apendices/bloques_correlacion_train.csv`.

Cuadro A.1: Los 13 bloques de variables redundantes ($|r| > 0,95$ sobre 500.000 flujos del conjunto de entrenamiento, semilla 42). Texto tomado literalmente de GRUPOS_CORRELACIONADOS en `src/features.py`.

#	Representante conservada	Variables eliminadas	Razón de la elección
1	Total Fwd Packets	Total Backward Packets, Subflow Fwd Packets, Subflow Bwd Packets, Subflow Bwd Bytes, Total Length of Bwd Packets, act_data_pkt_fwd	Todas cuentan volumen de la conversación y se mueven juntas; 'paquetes enviados por el origen' es el conteo más directo. Las 'Subflow *' son alias exactos de los totales.
2	Flow IAT Max	Fwd IAT Max, Idle Max, Idle Mean, Idle Min	Todas capturan el mayor silencio dentro de la conversación; 'Flow IAT Max' es la versión global.
3	Bwd Packet Length Mean	Avg Bwd Segment Size, Bwd Packet Length Max, Bwd Packet Length Std	Tamaño de los paquetes de respuesta; 'Avg Bwd Segment Size' es un alias exacto de la media, y máximo/desviación van casi pegados a ella.
4	Total Length of Fwd Packets	Subflow Fwd Bytes	Alias exacto ($r=1.000$); se conserva el nombre directo: bytes enviados por el origen.
5	Fwd Packet Length Mean	Avg Fwd Segment Size	Alias exacto ($r=1.000$) del tamaño medio de paquete de ida.
6	Fwd URG Flags	CWE Flag Count	Idénticas en los datos ($r=1.000$); se conserva la bandera TCP estándar (URG), mejor documentada que el conteo 'CWE'.
7	SYN Flag Count	Fwd PSH Flags	Idénticas en los datos ($r=1.000$); SYN (inicio de conexión) es clave para explicar escaneos de puertos.
8	RST Flag Count	ECE Flag Count	Idénticas en los datos ($r=1.000$); RST (conexión rechazada/cortada) es la más interpretable.
9	Flow Duration	Fwd IAT Total	$r=0.999$; la duración de la conversación es la feature más fácil de explicar del dataset.
10	Packet Length Mean	Average Packet Size	$r=0.998$; son dos fórmulas casi iguales del tamaño medio de paquete.
11	Max Packet Length	Packet Length Std	$r=0.984$; el máximo es más directo de explicar que la desviación.
12	Flow Packets/s	Fwd Packets/s	$r=0.983$; se conserva el ritmo global de la conversación.
13	Fwd Packet Length Max	Fwd Packet Length Std	$r=0.969$; mismo criterio: máximo antes que desviación.

B. Configuración de parámetros

La retroalimentación pidió reportar la configuración de los parámetros en lugar de decir que se usaron “valores estándar”. Nada de lo que sigue se escribió de memoria: las tablas de los modelos desplegados se generaron leyendo `get_params()` y los atributos ajustados de los archivos de `models/`; las del protocolo, instanciando los estimadores con la misma función del código que los construyó en los experimentos (`construir_pipeline` en `src/experimentos.py`) y leyendo las constantes de `src/`. El script que lo hace es `reports/apendices/generar_apendices.py`; su salida tabular completa (264 filas) está en `reports/apendices/parametros_modelos.csv`.

Qué significa “explícito”. Un parámetro se marca como explícito cuando está *escrito en la llamada del código fuente* (el script lee las llamadas de `src/` con el analizador sintáctico de Python y las registra en `llamadas_en_codigo.csv`), aunque el valor escrito coincida con el valor por defecto de la librería: es el caso de `n_neighbors=20` en el Local Outlier Factor, de

`n_splits=5` en la validación cruzada y de `n_repeats=5` en la importancia por permutación, que se señalan como “explícito (igual al defecto)”. “Por defecto” significa que el código no lo escribe y rige el valor de la librería.

B.1. Iteraciones reales y parada temprana

Es el punto donde la lectura del código corrige al documento anterior, que decía que el modelo de árboles “usa 200 iteraciones”. En realidad `max_iter=200` es un **máximo**. `HistGradientBoosting` trae `early_stopping='auto'`, que se activa cuando hay más de 10.000 observaciones —siempre, en este proyecto—: reserva el 10 % del tramo de entrenamiento (`validation_fraction=0.1`) como validación interna, ajusta los árboles con el 90 % restante y se detiene cuando la pérdida en esa validación no mejora en más de `tol = 10-7` durante `n_iter_no_change = 10` iteraciones seguidas. El Cuadro B.1 resume lo que ocurrió con los modelos serializados (entrenados con todo el conjunto de entrenamiento, los mismos que usa el tablero). Los modelos de cada partición de la validación cruzada no se conservaron (solo sus métricas), por lo que su número exacto de iteraciones no se reporta; usaron la misma configuración y el mismo techo.

Cuadro B.1: Iteraciones realmente usadas por los modelos serializados, léidas del atributo `n_iter_`. “Mejor iteración” es la de menor pérdida en la validación interna; el entrenamiento se detuvo 10 iteraciones después.

Modelo	Techo	Parada temprana	Iteraciones usadas	Mejor iteración	Árboles
Clasificador multiclase	200	activa	43	33	473 (11 por iteración)
Clasificador binario	200	activa	110	100	110

B.2. Modelos desplegados (leídos de `models/`)

Los dos cuadros siguientes listan *todos* los parámetros de los objetos serializados. Los dos clasificadores comparten exactamente la misma configuración (se verifica por código), así que van en un solo cuadro con los atributos ajustados de cada uno en columnas. Los únicos parámetros escritos en el código son: en los clasificadores, `class_weight='balanced'`, `max_iter=200` y `random_state=42`; en el Isolation Forest, `n_estimators=200`, `random_state=42` y `n_jobs=-1` (este último solo reparte el cómputo entre núcleos; no cambia el resultado). En el Isolation Forest, `contamination='auto'` queda en su valor por defecto porque *no se usa*: el corte lo fija el percentil de los scores del lunes (Sección 5.1); y `max_samples='auto'` significa 256 flujos por árbol, el valor que la librería fija para ese modo. Los dos `StandardScaler` del proyecto —el del detector, serializado en `models/`, y el que precede a la regresión logística dentro de cada partición— se instancian sin argumentos y quedan enteramente por defecto (`copy=True`, `with_mean=True`, `with_std=True`). El del detector se ajustó con los 394.236 flujos benignos del lunes (`n_samples_seen_`) y 48 variables.

Cuadro B.2: Clasificadores multiclase y binario (misma configuración): `HistGradientBoostingClassifier`, 21 parámetros. En **negrilla**, los escritos en la llamada del código (aunque coincidan con el valor por defecto); el resto quedó en el valor por defecto de scikit-learn 1.9.0.

Parámetro	Valor usado	Valor por defecto	Origen
<code>categorical_features</code>	<code>'from_dtype'</code>	<code>'from_dtype'</code>	por defecto
<code>class_weight</code>	<code>'balanced'</code>	<code>None</code>	explícito
<code>early_stopping</code>	<code>'auto'</code>	<code>'auto'</code>	por defecto

Parámetro	Valor usado	Valor por defecto	Origen
interaction_cst	None	None	por defecto
l2_regularization	0.0	0.0	por defecto
learning_rate	0.1	0.1	por defecto
loss	'log_loss'	'log_loss'	por defecto
max_bins	255	255	por defecto
max_depth	None	None	por defecto
max_features	1.0	1.0	por defecto
max_iter	200	100	explícito
max_leaf_nodes	31	31	por defecto
min_samples_leaf	20	20	por defecto
monotonic_cst	None	None	por defecto
n_iter_no_change	10	10	por defecto
random_state	42	None	explícito
scoring	'loss'	'loss'	por defecto
tol	1e-07	1e-07	por defecto
validation_fraction	0.1	0.1	por defecto
verbose	0	0	por defecto
warm_start	False	False	por defecto
<i>Atributos ajustados</i>	Multiclase	Binario	
do_early_stopping_	True	True	
n_iter_ (iteraciones realmente usadas)	43	110	
n_trees_per_iteration_ árboles totales	11	1	
mejor iteración en la validación interna	473	110	
n_features_in_	33	100	
clases	48	48	
	11	2	

Cuadro B.3: Detector de anomalías: bosque: IsolationForest, 9 parámetros. En **negrilla**, los escritos en la llamada del código (aunque coincidan con el valor por defecto); el resto quedó en el valor por defecto de scikit-learn 1.9.0.

Parámetro	Valor usado	Valor por defecto	Origen
bootstrap	False	False	por defecto
contamination	'auto'	'auto'	por defecto
max_features	1.0	1.0	por defecto
max_samples	'auto'	'auto'	por defecto
n_estimators	200	100	explícito
n_jobs	-1	None	explícito
random_state	42	None	explícito
verbose	0	0	por defecto
warm_start	False	False	por defecto
<i>Atributos ajustados (leídos del modelo entrenado)</i>			
max_samples_ (flujos por árbol)	256		
n_features_in_ umbral cuantil 0,5% / 1% / 2%	48		
	-0,6369 / -0,6140 / -0,5870		

B.3. Estimadores del protocolo que no se despliegan

La regresión logística (referencia lineal), las dos técnicas de remuestreo y el Local Outlier Factor no viven en `models/`. Sus parámetros se leen instanciándolos con el mismo código que los usó. Dos precisiones. En la regresión logística y en el HistGradientBoosting de la matriz de la

Fase 3, `class_weight` está escrito en la llamada (`class_weight=peso`) y vale `None` salvo en la columna “pesos de clase”, donde vale `'balanced'`: por eso figura como explícito aunque en tres de las cuatro columnas su valor sea el por defecto. Y en la regresión logística, `penalty` aparece como `'deprecated'` porque scikit-learn 1.8 retiró ese parámetro; la regularización efectiva es L2 (`l1_ratio=0.0`) con `C=1.0` y el solucionador `lbfgs`, todo por defecto.

Cuadro B.4: Regresión logística (referencia lineal): `LogisticRegression`, 14 parámetros. En **negrilla**, los escritos en la llamada del código (aunque coincidan con el valor por defecto); el resto quedó en el valor por defecto de scikit-learn 1.9.0.

Parámetro	Valor usado	Valor por defecto	Origen
<code>C</code>	1.0	1.0	por defecto
<code>class_weight</code>	<code>None</code> <code>'balanced'</code> (columna “pesos”)	<code>None</code>	explícito (según la técnica)
<code>dual</code>	<code>False</code>	<code>False</code>	por defecto
<code>fit_intercept</code>	<code>True</code>	<code>True</code>	por defecto
<code>intercept_scaling</code>	1	1	por defecto
<code>l1_ratio</code>	0.0	0.0	por defecto
<code>max_iter</code>	1000	100	explícito
<code>n_jobs</code>	<code>None</code>	<code>None</code>	por defecto
<code>penalty</code>	<code>'deprecated'</code>	<code>'deprecated'</code>	por defecto
<code>random_state</code>	42	<code>None</code>	explícito
<code>solver</code>	<code>'lbfgs'</code>	<code>'lbfgs'</code>	por defecto
<code>tol</code>	0.0001	0.0001	por defecto
<code>verbose</code>	0	0	por defecto
<code>warm_start</code>	<code>False</code>	<code>False</code>	por defecto

Cuadro B.5: SMOTE (`sampling_strategy` es la función `estrategia_smote`: eleva a 20.000 las clases con menos casos que eso): `SMOTE`, 3 parámetros. En **negrilla**, los escritos en la llamada del código (aunque coincidan con el valor por defecto); el resto quedó en el valor por defecto de imbalanced-learn 0.14.2.

Parámetro	Valor usado	Valor por defecto	Origen
<code>k_neighbors</code>	5	5	por defecto
<code>random_state</code>	42	<code>None</code>	explícito
<code>sampling_strategy</code>	función <code>estrategia_smote</code>	<code>'auto'</code>	explícito

Cuadro B.6: Submuestreo aleatorio (`sampling_strategy` es la función `estrategia_submuestreo`: reduce BENIGN a 200.000 casos): `RandomUnderSampler`, 3 parámetros. En **negrilla**, los escritos en la llamada del código (aunque coincidan con el valor por defecto); el resto quedó en el valor por defecto de imbalanced-learn 0.14.2.

Parámetro	Valor usado	Valor por defecto	Origen
<code>random_state</code>	42	<code>None</code>	explícito
<code>replacement</code>	<code>False</code>	<code>False</code>	por defecto
<code>sampling_strategy</code>	función <code>estrategia_submuestreo</code>	<code>'auto'</code>	explícito

Cuadro B.7: Local Outlier Factor (contraste; entrenado con una submuestra de 50.000 flujos benignos del lunes): `LocalOutlierFactor`, 9 parámetros. En **negrilla**, los escritos en la llamada del código (aunque coincidan con el valor por defecto); el resto quedó en el valor por defecto de scikit-learn 1.9.0.

Parámetro	Valor usado	Valor por defecto	Origen
<code>algorithm</code>	'auto'	'auto'	por defecto
<code>contamination</code>	'auto'	'auto'	por defecto
<code>leaf_size</code>	30	30	por defecto
<code>metric</code>	'minkowski'	'minkowski'	por defecto
<code>metric_params</code>	None	None	por defecto
<code>n_jobs</code>	-1	None	explícito
<code>n_neighbors</code>	20	20	explícito (igual al defecto)
<code>novelty</code>	True	False	explícito
<code>p</code>	2	2	por defecto

Cuadro B.8: Parámetros del protocolo que no viven en los modelos serializados.

Componente	Objeto	Valores fijados	Dónde
Partición única	<code>train_test_split</code>	<code>test_size=0.2; stratify=Label;</code> <code>random_state=42</code>	<code>src/split.py</code>
Validación cruzada	<code>StratifiedKFold</code>	<code>n_splits=5</code> (igual al defecto); <code>shuffle=True;</code> <code>random_state=42</code>	<code>src/experimentos.py</code> , <code>src/interpretabilidad.py</code> , <code>notebooks/02_baseline.ipynb</code>
Clasificador trivial (línea base)	<code>DummyClassifier</code>	<code>strategy='most_frequent'</code>	<code>notebooks/02_baseline.ipynb</code>
Importancia por permutación	<code>permutation_importance</code>	<code>scoring='f1_macro';</code> <code>n_repeats=5</code> (igual al defecto); <code>random_state=42;</code> <code>n_jobs=-1</code> ; una sola partición (300.000 flujos de la validación de la primera)	<code>src/interpretabilidad.py</code>
Modelo explicado por permutación	<code>HistGradientBoostingClassifier</code>	binario; misma configuración del Cuadro B.2; ajustado con el 80 % de la primera partición	<code>src/interpretabilidad.py</code>
Explicación por alerta	<code>shap.TreeExplainer</code>	sobre el clasificador multiclase serializado; parámetros por defecto	<code>app/pantallas.py</code>
Umbral de operación de Bot	regla	alarma de Bot solo si $P(\text{Bot}) \geq 0,999$; si no, la segunda clase más probable; fijado con probabilidades out-of-fold del entrenamiento	<code>src/evaluacion_final.py</code>
Cuantiles del detector	constante	0,5 %; 1 %; 2 % de los scores del lunes (1 % = referencia)	<code>src/no_supervisado.py</code> , <code>src/evaluacion_final.py</code>
Variable identificadora retirada (prueba del puerto)	constante	<code>Destination Port</code>	<code>src/interpretabilidad.py</code>
Selección de variables	constante	$ r > 0,95$ sobre 500.000 flujos del entrenamiento (semilla 42); una representante por bloque	<code>src/features.py</code>
Semilla única y versiones	constante	<code>RANDOM_STATE = 42;</code> scikit-learn 1.9.0; Python 3.13; imbalanced-learn 0.14.2	<code>src/config.py</code> , <code>models/metadatos.json</code> , <code>requirements*.txt</code>